`finally`, `write_env`, and `unstoppable` Sender Adaptors

Document #:	P3284R0
Date:	2024-05-15
Project:	Programming Language C++
Audience:	LEWG Library Evolution

Reply-to:	Eric Niebler <eric.niebler@gmail.com>

Contents

§ 1 Introduction

This paper proposes to add three new sender adaptor algorithms to the `std::execution` namespace, targetting C++26: `finally`, `write_env`, and `unstoppable`. These adaptors were originally proposed as part of [P3175R0] but have been split out into their own paper so that the higher priority items in P3175 can advance more quickly.

§ 2 Executive Summary

Below are the specific changes this paper proposes:

1. Add a new uncustomizable adaptor `write_env` for writing values into the receiver's execution environment, and rename `read` to `read_env` ("`read`" being too vague and something of a land-grab). `write_env` is used in the implementation of the new `on` algorithm ([P3175R1]) and can simplify the specification of the `let_` algorithms.

2. Add an uncustomizable `unstoppable` adaptor that is a trivial application of `write_env`: it sets the current stop token in the receiver's environment to a `never_stop_token`. `unstoppable` is used in the re-specification of the `schedule_from` algorithm.

3. Generalize the specification for `schedule_from` to take two senders instead of a sender and a scheduler, name it `finally`, and make it uncustomizable. Specify the default implementation of `schedule_from(sch, snd)` as `finally(snd, unstoppable(schedule(sch)))`.

§ 3 Description

[P3175R0] proposes some changes to the `std::execution::on` algorithm, the specification of which was made simpler by the addition of some additional adaptors. Those adaptors were general and useful in their own right, so P3175R0 suggested they be added to `std::execution` proper. The conservative approach is to keep them exposition-only, and [P3175R1] proposes just that.

The author still feels like those adaptors, which were removed from P3175, are worthy of standardization. This paper proposes adding them.

The adaptors in question are as follows:

§ 3.1 `write_env`

A receiver has an associated "execution environment", which is an unstructured, queryable key/value store. It is used to pass implicit parameters from parent operations to their children. It is occasionally useful for a sender adaptor to explicitly mutate the key/value store so that child operations see different values for environment queries. The `write_env` sender adaptor is used for that purpose.

`write_env` is a customization point object, although it is not actually customizable. It accepts a sender `sndr` and an execution environment `env`, and it returns a new sender that stores `sndr` and `env`. When that sender is connected to a receiver `rcvr`, it returns the result of connecting `sndr` with a receiver that adapts `rcvr`. The environment of that adapted receiver is the result of joining `env` with `rcvr`'s environment. The two environments are joined such that, when the joined environment is queried, `env` is queried first, and if `env` doesn't have a value for that query, the result of `get_env(rcvr)` is queried.

§ 3.1.1 Example: `write_env`

One example of where `write_env` might be useful is to specify an allocator to be used by child operations. The code might look like this:

```
// Turn a query object and a value into a queryable environment:
template <class Query, class Value>
struct with : Query {
  Value value;
  auto query(Query) const { return value; }
};

// Adapts a sender so that it can use the given allocator:
struct with_allocator_t {
  template <std::execution::sender Sndr, class Alloc>
  auto operator()(Sndr sndr, Alloc alloc) const {
         return  std::execution::write_env(sndr,  with{std::get_allocator,
        alloc});
  }
};

constexpr with_allocator_t with_allocator{};
```

The `with_allocator` adaptor might be used to parameterize senders produced by a third-party library as follows:

```
namespace ex = std::execution;

// This returns a sender that does some piece of asynchronous work
// created by a third-party library, but parameterized with a custom
// allocator.
ex::sender auto make_async_work_with_alloc() {
  ex::sender auto work = third_party::make_async_work();

  return with_allocator(std::move(work), custom_allocator());
}
```

The sender returned by `third_party::make_async_work` might query for the allocator and use it to do allocations:

```
namespace third_party {
  namespace ex = std::execution;

  // A function that returns a sender that generates data on a special
  // execution context, populate a std::vector with it, and then completes
  // by sending the vector.
  constexpr auto _populate_data_vector =
    []<class Allocator>(Allocator alloc) {
      // Create an empty vector of ints that uses a specified allocator.
           using  IntAlloc  =  std::allocator_traits<Allocator>::template
        rebind_alloc<int>;
      auto data = std::vector<int, IntAlloc>{IntAlloc{std::move(alloc)}};

      // Create some work that generates data and fills in the vector.
      auto work = ex::just(std::move(data))
```

```
        | ex::then([](auto data) {
            // Generate the data and fill in the vector:
            data.append_range(third_party::make_data())
            return data;
          });

      // Execute the work on a special third_party execution context:
      // (This uses the `on` as specified in P3175.)
      return ex::on(third_party_scheduler(), std::move(work));
    };

      //   A   function   that   returns   the   sender   produced   by
        `_populate_data_vector`,
  // parameterized by an allocator read out of the receiver's environment.
  ex::sender auto make_async_work() {
    return ex::let_value(
         // This reads the allocator out of the receiver's execution
        environment.
      ex::read_env(std::get_allocator),
      _populate_data_vector
    );
  }
}
```

§ 3.2 `unstoppable`

The `unstoppable` sender adaptor is a trivial application of `write_env` that modifies a sender so that it no longer responds to external stop requests. That can be of critical importance when the successful completion of a sender is necessary to ensure program correctness, *e.g.*, to restore an invariant.

The `unstoppable` adaptor might be implemented as follows:

```
inline constexpr struct unstoppable-t {
  template <sender Sndr>
  auto operator()(Sndr sndr) const {
    return write_env(std::move(sndr), never_stop_token());
  }

  auto operator()() const {
    return write_env(never_stop_token());
  }
} unstoppable {};
```

The section describing the `finally` adaptor will give a motivating example that makes use of `unstoppable`.

§ 3.3 `finally`

The C++ language lacks direct support for asynchronous destruction; that is, there is no way to say, “After this asynchronous operation, unconditionally run another asynchronous operation, regardless of how the first one completes.” Without this capability, there is no

native way to have “async RAII”: the pairing the asynchronous acquisition of a resource with its asynchronous reclaimation.

The `finally` sender adaptor captures the “async RAII” pattern in the sender domain. `finally` takes two senders. When connected and started, the `finally` sender connects and starts the first sender. When that sender completes, it saves the asynchronous result and then connects and starts the second sender. If the second sender completes successfully, the results from the first sender are propagated. Otherwise, the results from the second sender are propagated.

There is a sender in [P2300R9] very much like `finally` as described above: `schedule_from`. The only meaningful difference is that in `schedule_from`, the “second sender” is always the result of calling `schedule` on a scheduler. With `finally`, the default implementation of `schedule_from` is trivial:

```
template <sender Sndr, scheduler Sched>
auto default-schedule-from-impl(Sndr sndr, Sched sched) {
  return finally(std::move(sndr), unstoppable(schedule(sched)));
}
```

This paper proposes repurposing the wording of `schedule_from` to specify `finally`, and then specifying `schedule_from` in terms of `finally` and `unstoppable`.

§ 3.3.1 Example: `finally`

In the following example, some asynchronous work must temporarily break a program invariant. It uses `unstoppable` and `finally` to restore the invariant.

```
  namespace ex = std::execution;

  ex::sender auto break_invariants(auto&... values);
  ex::sender auto restore_invariants(auto&... values);

  // This function returns a sender adaptor closure object. When applied
        to
  // a sender, it returns a new sender that breaks program invariants,
  // munges the data, and restores the invariants.
  auto safely_munge_data( ) {
    return ex::let_value( [](auto&... values) {
        return break_invariants(values...)
           | ex::then(do_munge) // the invariants will be restored even if
        `do_munge` throws
          | ex::finally(ex::unstoppable(restore_invariants(values...)));
    } );
  }

  auto sndr = ...;
  scope.spawn( sndr | safely_munge_data() ); // See `counting_scope` from
        P3149R2
```

§ 4 Discussion

There are a number of design considerations for the `finally` algorithm. The following questions have been brought up during LEWG design review:

§ 4.1 Should `finally` apply `unstoppable` by default to its second argument?

The observation was made that, since `finally` will often be used to do some cleanup operation or to restore an invariant, that operation should not respond to external stop requests, so `unstoppable` should be the default. It's a reasonable suggestion. Of course, there would need to be a way to override the default and allow the cleanup action to be canceled, and it isn't clear what the syntax for that would be. Another adaptor called `stoppable_finally`?

It is worth nothing that `unstoppable` has uses besides `finally`, so it arguably should exist regardless of what the default behavior of `finally` is. Given that `unstoppable` should exist anyway, and that its behavior is pleasantly orthogonal to `finally`, the authors decided to keep them separate and let users combine them how they like.

§ 4.2 Is there a different design that better captures the "async RAII" intent?

Undoubtedly, the answer is "yes." There are probably several such designs. One design that has been explored by Kirk Shoop is the so-called "async resource".

In Kirk's design, an async resource has three basis operations: `open`, `run`, and `close`, each of which is asynchronous; that is, they all return senders. When `open` completes, it does so with a handle to the resource. The handle lets you interact with the resource. Calling `close` on the handle ends the lifetime of the async resource.

Some resources, like a resource-ified `run_loop`, need to be driven in order to function. That is what the `run` operation is for. `run` and `open` can be started simultaneously. In the case of a `run_loop` resource, `run` drives the loop, `open` returns a `run_loop` scheduler that also implements `close`, and `close` would call `finish` on the `run_loop`. The `run` sender will complete when all the work queued on the `run_loop` has finished.

Similarly, an async scope *a la* [P3149R2]'s `counting_scope` can be given the async resource treatment. Multiple such async resources can be used in tandem, as in the following example:

```
// In this example, run_loop and counting_scope implement the async
// resource concept.
run_loop loop;
counting_scope scope;

auto use = ex::when_all(ex::open(loop), ex::open(scope))
         | ex::let_value([](auto h_loop, auto h_scope) {
             // The lifetimes of the async resources have begun when
             // `open` completes.
```

```
            // spawn 1000 tasks on the run_loop in the counting_scope.
            for (int i = 0; i < 1000; ++i) {
              auto work = ex::just() | ex::then([=]{ do_work(i);});
              h_scope.spawn(ex::on(h_loop, std::move(work)));
            }

            // The lifetime of the resources end when their close senders
            // start executing.
            return ex::when_all(ex::close(h_loop), ex::close(h_scope));
          });

// Drive the resources while using them. (The run_loop is driven from
// a separate worker thread.)
auto work = ex::when_all(
              use,
              ex::on(worker, ex::run(loop)),
              ex::run(scope)
            );

// Launch it all and wait for it to complete
ex::sync_wait(std::move(work));
```

This design nicely captures the "async RAII" pattern. A type modeling the async resource concept is like an async class with an async constructor and an async destructor. Instead of using `finally`, a user can implement a type that satisfies the async resource concept.

Although there are times when it is appropriate to model the async resource concept, doing so is certainly more work than just using `finally`. One can think of `finally` as an *ad hoc* form of async RAII. To draw an analogy, `finally` is to async resource what `scope_guard` is to custom RAII wrappers like `unique_ptr`. That is no diss on `scope_guard`; it has its place!

So too does `finally` in the authors' opinion. It captures a common pattern quite simply, and is not a far departure from what is in P2300 already. An async resource abstraction is a much heavier lift from a standardization point of view. Pursuing that design instead of `finally` risks missing the C++26 boat, leaving users without a standard way to reliably clean up asynchronous resources.

In the end, the authors expect that we will have both, just as many codebases make use of both `scope_guard` and `unique_ptr`.

§ 5 Proposed Wording

[Editor's note: The wording in this section is based on [P2300R9] with the additions of [P2855R1] and [P3175R1].]

[Editor's note: Change [exec.syn] as follows:]

```
  inline constexpr unspecified read_env{};
...
```

```
  struct start_on_t;
  struct ~~transfer_t~~continue_on_t;
  struct on_t;
  struct schedule_from_t;
...

  inline constexpr unspecified write_env{};
  inline constexpr unspecified unstoppable{};
  inline constexpr start_on_t start_on{};
  inline constexpr ~~transfer_t transfer~~continue_on_t continue_on{};
  inline constexpr on_t on{};
  inline constexpr unspecified finally{};
  inline constexpr schedule_from_t schedule_from{};
```

[Editor's note: Change subsection "`execution::read` [exec.read]" to "`execution::read_env` [exec.read.env]", and within that subsection, replace every instance of "`read`" with "`read_env`".]

[Editor's note: Replace all instances of "*`write-env`*" with "`write_env`". After [exec.adapt.objects], add a new subsection "`execution::write_env` [exec.write.env]" and move the specification of the exposition-only *`write-env`* from [exec.snd.general]/p3.15 into it with the following modifications:]

(34.9.11.?) **`execution::write_env` [exec.write.env]**

1. ~~*`write-env`*~~`write_env` is ~~an exposition-only~~a sender adaptor that accepts a sender and a queryable object, and that returns a sender that, when connected with a receiver `rcvr`, connects the adapted sender with a receiver whose execution environment is the result of joining the `queryable` ~~argument env~~object to the result of `get_env(rcvr)`.

2. Let *`write-env-t`* be an exposition-only empty class type.

3. *Returns:* *`make-sender`*`(`*`make-env-t`*`(), std::forward<Env>(env), std::forward<Sndr>(sndr))`.

2. `write_env` is a customization point object. For some subexpressions `sndr` and `env`, if `decltype((sndr))` does not satisfy `sender` or if `decltype((env))` does not satisfy `queryable`, the expression `write_env(sndr, env)` is ill-formed. Otherwise, it is expression-equivalent to *`make-sender`*`(write_env, env, sndr)`.

3. ~~*Remarks:*~~ The exposition-only class template *`impls-for`* ([exec.snd.general]) is specialized for ~~*`write-env-t`*~~`write_env` as follows:

```
template<>
struct  impls-for<~~write-env-t~~decayed-typeof<write_env>>  :  default-
        impls {
  static constexpr auto get-env =
    [](auto, const auto& state, const auto& rcvr) noexcept {
      return JOIN-ENV(state, get_env(rcvr));
```

```
    };
};
```

[Editor's note: After [exec.write.env], add a new subsection "`execution::unstoppable` [exec.unstoppable]" as follows:]

(34.9.11.?) **`execution::unstoppable` [exec.unstoppable]**

1. `unstoppable` is a sender adaptor that connects its inner sender with a receiver that has the execution environment of the outer receiver but with a `never_stop_token` as the value of the `get_stop_token` query.

2. For a subexpression `sndr`, `unstoppable(sndr)` is expression equivalent to `write_env(sndr, MAKE-ENV(get_stop_token, never_stop_token{}))`.

[Editor's note: Change subsection "`execution::schedule_from` [exec.schedule.from]" to "`execution::finally` [exec.finally]", change every instance of "`schedule_from`" to "`finally`" and "`schedule_from_t`" to "*`decayed-typeof`*`<finally>`", and change the subsection as follows:]

(34.9.11.5) **`execution::finally` [exec.finally]**

[Editor's note: Replace paragraphs 1-3 with the following:]

1. `finally` is a sender adaptor that starts one sender unconditionally after another sender completes. If the second sender completes successfully, the `finally` sender completes with the async results of the first sender. If the second sender completes with error or stopped, the async results of the first sender are discarded, and the `finally` sender completes with the async results of the second sender. [*Note:* It is similar in spirit to the `try`/`finally` control structure of some languages. — *end note*]

2. The name `finally` denotes a customization point object. For some subexpressions `try_sndr` and `finally_sndr`, if `try_sndr` or `finally_sndr` do not satisfy `sender`, the expression `finally(try_sndr, finally_sndr)` is ill-formed; otherwise, it is expression-equivalent to *`make-sender`*`(finally, {}, try_sndr, finally_sndr)`.

3. Let `CS` be a specialization of `completion_signatures` whose template parameters are the pack `Sigs`. Let *`VALID-FINALLY`*`(CS)` be `true` if and only if there is no type in `Sigs` of the form `set_value_t(Ts...)` for which `sizeof...(Ts)` is greater than `0`. Let `F` be `decltype((finally_sndr))`. If `sender_in<F>` is `true` and *`VALID-FINALLY`*`(completion_signatures_of_t<F>)` is `false`, the program is ill-formed.

4. The exposition-only class template *`impls-for`* ([exec.snd.general]) is specialized for `finally` as follows:

```
namespace std::execution {
  template<>
```

```
  struct impls-for<decayed-typeof<finally>> : default-impls {
    static constexpr auto get-attrs = see below;
    static constexpr auto get-state = see below;
    static constexpr auto complete = see below;
  };
}
```

1. The member *impls-for*<*decayed-typeof*<finally>>::*get-attrs* is initialized with a callable object equivalent to the following lambda:

```
[](const auto& data, const auto& child) noexcept ->
        decltype(auto) {
        return    JOIN-ENV(SCHED-ATTRS(data),    FWD-
        ENV(get_env(child)));
}
```

```
[](auto,  const  auto&  tsndr,  const  auto&  fsndr)
        noexcept -> decltype(auto) {
     return   JOIN-ENV(FWD-ENV(get_env(fsndr)),   FWD-
        ENV(get_env(tsndr)));
}
```

2. The member *impls-for*<*decayed-typeof*<finally>>::*get-state* is initialized with a callable object equivalent to the following lambda:

```
[]<class Sndr, class Rcvr>(Sndr&& sndr, Rcvr& rcvr)
    requires sender_in<child-type<Sndr, 0>, env_of_t<Rcvr>>
        &&
      sender_in<child-type<Sndr, 1>, env_of_t<Rcvr>> &&
            VALID-FINALLY(completion_signatures_of_t<child-
        type<Sndr, 1>, env_of_t<Rcvr>>) {
  return apply(
      [&]<class Sch, class Child>(auto, Sch sch, Child&&
        child)
       [&]<class TSndr, class FSndr>(auto, auto, TSndr&&
        tsndr, FSndr&& fsndr) {
      using variant-type = see below;
      using receiver-type = see below;
                              using    operation-type    =
        connect_result_t<schedule_result_tFSndr, receiver-
        type>;

      struct state-type {
        Rcvr& rcvr;
        variant-type async-result;
        operation-type op-state;

          explicit state-type(Sch schFSndr&& fsndr, Rcvr&
        rcvr)
          : rcvr(rcvr)
                                                ,    op-
        state(connect(schedule(sch)std::forward<FSndr>
        (fsndr), receiver-type{{}, this})) {}
      };

          return  state-type{schstd::forward<FSndr>(fsndr),
        rcvr};
```

```
    },
    std::forward<Sndr>(sndr));
}
```

1. The local class *state-type* is a structural type.

2. Let `Sigs` be a pack of the arguments to the `completion_signatures` specialization named by `completion_signatures_of_t<~~Child~~TSndr, env_of_t<Rcvr>>`. Let *as-tuple* be an alias template that transforms a completion signature `Tag(Args...)` into the `tuple` specialization *decayed-tuple*`<Tag, Args...>`. Then *variant-type* denotes the type `variant<monostate, `*as-tuple*`<Sigs>...>`, except with duplicate types removed.

3. Let *receiver-type* denote the following class:

```
namespace std::execution {
  struct receiver-type {
    using receiver_concept = receiver_t;
    state-type* state; // exposition only

          ~~Rcvr&& base() && noexcept { return
        std::move(state->rcvr); }~~
      ~~const Rcvr& base() const & noexcept { return
        state->rcvr; }~~

    void set_value() && noexcept {
      visit(
         [this]<class Tuple>(Tuple& result) noexcept -
        > void {
          if constexpr (!same_as<monostate, Tuple>) {
            auto& [tag, ...args] = result;
                          tag(std::move(state->rcvr),
        std::move(args)...);
          }
        },
        state->async-result);
    }

    template<class Error>
    void set_error(Error&& err) && noexcept {
          execution::set_error(std::move(state->rcvr),
        std::forward<Error>(err));
    }

    void set_stopped() && noexcept {
      execution::set_stopped(std::move(state->rcvr));
    }

    decltype(auto) get_env() const noexcept {
             return  FWD-ENV(execution::get_env(state-
        >rcvr));
    }
```

```
  };
}
```

3. The member *impls-for*<*decayed-typeof*<finally>>::*complete* is initialized with a callable object equivalent to the following lambda:

```
[]<class Tag, class... Args>(auto, auto& state, auto& rcvr,
        Tag, Args&&... args) noexcept -> void {
  using result_t = decayed-tuple<Tag, Args...>;
               constexpr       bool       nothrow       =
        is_nothrow_constructible_v<result_t,          Tag,
        Args...>;

  TRY-EVAL(std::move(rcvr), [&]() noexcept(nothrow) {
      state.async-result.template  emplace<result_t>(Tag(),
        std::forward<Args>(args)...);
  }());

  if (state.async-result.valueless_by_exception())
    return;
  if (state.async-result.index() == 0)
    return;

  start(state.op-state);
};
```

[Editor's note: Remove paragraph 5, which is about the requirements on customizations of the algorithm; `finally` cannot be customized.]

[Editor's note: Insert a new subsection "`execution::schedule_from` [exec.schedule.from]" as follows:]

(34.9.11.?) **`execution::schedule_from` [exec.schedule.from]**

[Editor's note: These three paragraphs are taken unchanged from P2300R8.]

1. `schedule_from` schedules work dependent on the completion of a sender onto a scheduler's associated execution resource. [*Note:* `schedule_from` is not meant to be used in user code; it is used in the implementation of `continue_on`. — *end note*]

2. The name `schedule_from` denotes a customization point object. For some subexpressions `sch` and `sndr`, let `Sch` be `decltype((sch))` and `Sndr` be `decltype((sndr))`. If `Sch` does not satisfy `scheduler`, or `Sndr` does not satisfy `sender`, `schedule_from` is ill-formed.

3. Otherwise, the expression `schedule_from(sch, sndr)` is expression-equivalent to:

```
transform_sender(
  query-or-default(get_domain, sch, default_domain()),
  make-sender(schedule_from, sch, sndr));
```

4. The exposition-only class template *impls-for* is specialized for `schedule_from_t` as follows:

```
template<>
struct impls-for<schedule_from_t> : default-impls {
  static constexpr auto get-attrs =
         [](const auto& data, const auto& child) noexcept ->
        decltype(auto) {
      return JOIN-ENV(SCHED-ATTRS(data), FWD-ENV(get_env(child)));
    };
};
```

5. Let `sndr` and `env` be subexpressions such that `Sndr` is `decltype((sndr))`. If *`sender-for`*`<Sndr, schedule_from_t>` is `false`, then the expression `schedule_from.transform_sender(sndr, env)` is ill-formed; otherwise, it is equal to:

```
auto&& [tag, sch, child] = sndr;
return finally(std::forward_like<Sndr>(child),
               unstoppable(schedule(std::forward_like<Sndr>(sch))));
```

[*Note:* This causes the `schedule_from(sch, sndr)` sender to become `finally(sndr, unstoppable(schedule(sch)))` when it is connected with a receiver with an execution domain that does not customize `schedule_from`. — *end note*]

[Editor's note: The following paragraph is taken unchanged from P2300R9.]

6. Let the subexpression `out_sndr` denote the result of the invocation `schedule_from(sch, sndr)` or an object copied or moved from such, and let the subexpression `rcvr` denote a receiver such that the expression `connect(out_sndr, rcvr)` is well-formed. The expression `connect(out_sndr, rcvr)` has undefined behavior unless it creates an asynchronous operation ([async.ops]) that, when started:

   — eventually completes on an execution agent belonging to the associated execution resource of `sch`, and

   — completes with the same async result as `sndr`.

[Editor's note: The following changes to the `let_*` algorithms are not strictly necessary; they are simplifications made possible by the addition of the `write_env` adaptor above.]

[Editor's note: Remove [exec.let]p5.1, which defines an exposition-only class *`receiver2`*.]

[Editor's note: Change [exec.let]p5.2.2 as follows:]

2. Let *`as-sndr2`* be an alias template such that *`as-sndr2`*`<Tag(Args...)>` denotes the type *`call-result-t`*`<decayed-typeof<write_env>, call-result-t<Fn, decay_t<Args>&...>, Env>`. Then *`ops2-variant-type`* denotes the type `variant<monostate, connect_result_t<`*`as-sndr2`*`<LetSigs>, ~~receiver2<~~Rcvr~~, Env>~~>...>`.

[Editor's note: Change [exec.let]p5.3 as follows:]

3. The exposition-only function template *let-bind* is ~~equal to~~as follows:

```
template<class State, class Rcvr, class... Args>
void let-bind(State& state, Rcvr& rcvr, Args&&... args) {
      auto&   args   =   state.args.emplace<decayed-tuple<Args...>>
        (std::forward<Args>(args)...);
     auto   sndr2   =   write_env(apply(std::move(state.fn),   args),
        std::move(state.env)); // see [exec.adapt.general]
  auto rcvr2 = receiver2{std::move(rcvr), std::move(state.env)};
     auto   mkop2   =   [&]   {   return   connect(std::move(sndr2),
        std::move(rcvr2)); };
     auto&   op2   =   state.ops2.emplace<decltype(mkop2())>(emplace-
        from{mkop2});
  start(op2);
}
```

§ 6 References

[P2300R9] Eric Niebler, Michał Dominiak, Georgy Evtushenko, Lewis Baker, Lucian Radu Teodorescu, Lee Howes, Kirk Shoop, Michael Garland, Bryce Adelstein Lelbach. 2024-04-02. `std::execution`.

https://wg21.link/p2300r9

[P2855R1] Ville Voutilainen. 2024-02-22. Member customization points for Senders and Receivers.

https://wg21.link/p2855r1

[P3149R2] Ian Petersen, Ján Ondrušek; Jessica Wong; Kirk Shoop; Lee Howes; Lucian Radu Teodorescu; 2024-03-20. async_scope — Creating scopes for non-sequential concurrency.

https://wg21.link/p3149r2

[P3175R0] Eric Niebler. 2024-03-14. Reconsidering the `std::execution::on` algorithm.

https://wg21.link/p3175r0

[P3175R1] Eric Niebler. Reconsidering the `std::execution::on` algorithm.

https://isocpp.org/files/papers/P3175R1.html